

Day 7: File Handling, Data Formats, Serialization & Relational Databases

Welcome to Day 7! Today we focus on essential data persistence mechanisms, file I/O streams, structured serialization formats, and relational database interaction in Python. We will cover:

- 1. **File Handling & Context Managers:** Safely opening, reading, writing, and appending files using the `with` statement.
- 2. **Structured Tabular Formats (CSV):** Parsing, writing, and transforming tabular rows with the `csv` module (`reader`, `writer`, `DictReader`, `DictWriter`).
- 3. **Hierarchical Data Formats (JSON):** Serializing in-memory structures and parsing configuration/data payloads with the `json` module.
- 4. **Object Serialization (pickle):** Preserving and restoring arbitrary in-memory Python object states to disk in binary format.
- 5. **Relational Databases & DB-API (sqlite3):** Establishing database connections, creating tables, executing parameterized queries, implementing complete CRUD operations, and managing ACID transactions.

Part 1: File Handling & Context Managers

1. The `open()` Function & File Access Modes

Python interacts with file streams on disk using the built-in `open(file, mode, encoding)` function.

Mode	Description	Initial Pointer	Behavior if File Exists	Behavior if File Missing
'r'	Read (default)	Start of file	Opens for reading	Raises <code>FileNotFoundError</code>
'w'	Write	Start of file	Overwrites / truncates existing content	Creates new file
'a'	Append	End of file	Preserves data; appends new content at the end	Creates new file
'r+'	Read & Write	Start of file	Allows reading and writing without truncation	Raises <code>FileNotFoundError</code>
'b'	Binary Mode	Start of file	Handles raw byte streams (e.g. <code>'rb'</code> , <code>'wb'</code>)	Dependent on combined mode

2. Context Managers (`with` Statement)

Manual file operations require explicit `file.close()` calls, which can be skipped if an unexpected exception occurs. The `with` statement guarantees deterministic resource cleanup by automatically closing file descriptors when the block exits.

```
# Safe writing with context manager
with open("records.txt", "w", encoding="utf-8") as f:
    f.write("Header: System Audit Log\n")
    f.writelines(["Entry 1: Service Initialized\n", "Entry 2: Ingestion In Progress\n"])

# Safe reading line-by-line (memory-efficient lazy iteration)
with open("records.txt", "r", encoding="utf-8") as f:
    for line_num, line in enumerate(f, start=1):
        print(f"[{line_num}] {line.strip()}")
```

[!TIP] Iterating over the file object directly (`for line in f:`) streams lines lazily into memory rather than loading the entire file at once, ensuring near-zero memory overhead on large files.

Part 2: Working with CSV and JSON

1. Tabular Data with the `csv` Module

The standard `csv` library provides `reader` and `writer` for list-based row processing, as well as `DictReader` and `DictWriter` for dictionary-based column mapping.

```
import csv

trainees = [
    {"id": 101, "name": "Arham", "track": "AI", "score": 94.5},
    {"id": 102, "name": "Lisa", "track": "BDA", "score": 88.0},
    {"id": 103, "name": "Vinod", "track": "AI", "score": 96.0}
]

# Write CSV with column headers
with open("trainees.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "name", "track", "score"])
    writer.writeheader()
    writer.writerows(trainees)

# Read CSV into dictionary mappings
with open("trainees.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['name']} ({row['track']}) -> Score: {float(row['score']):.1f}")
```

2. Hierarchical Payloads with the `json` Module

JSON (JavaScript Object Notation) is the standard format for API data interchange and structured configuration storage.

- **String conversion:** `json.dumps(obj)` (serialize to string), `json.loads(str)` (deserialize from string).
- **File I/O:** `json.dump(obj, file)` (write directly to file stream), `json.load(file)` (read directly from file stream).

```
import json

app_config = {
    "service": "CDAC Data Platform",
    "version": "2.4.0",
    "database": {"host": "localhost", "port": 5432, "ssl": True},
    "features": ["analytics", "caching", "auth"]
}

# Write formatted JSON to file
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(app_config, f, indent=4)

# Read JSON back into a Python dictionary
with open("config.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
    print("Database Host:", loaded["database"]["host"])
```

Part 3: Object Serialization with `pickle`

Python's `pickle` module converts complex in-memory Python objects (including custom class instances, functions, and nested structures) into binary representations for persistent disk storage.

```
import pickle

class TraineeEvaluation:
    def __init__(self, trainee_id, name, scores):
        self.trainee_id = trainee_id
        self.name = name
        self.scores = scores

    def compute_average(self):
        return sum(self.scores) / len(self.scores)

student = TraineeEvaluation(101, "Arham", [90, 95, 92])

# Serialize object to binary file
with open("evaluation.pkl", "wb") as f:
    pickle.dump(student, f)

# Deserialize object from binary file
with open("evaluation.pkl", "rb") as f:
```

```
restored = pickle.load(f)
print(f"{restored.name} Average: {restored.compute_average():.2f}")
```

[!CAUTION] The `pickle` module is not secure against erroneous or maliciously constructed data. Never unpickle data received from untrusted or unauthenticated clients.

Part 4: Relational Databases & SQLite (Python DB-API 2.0)

Python includes native support for SQLite through the `sqlite3` module, adhering to the PEP 249 DB-API 2.0 specification.

1. Core Workflow

1. **Connect:** Open a connection to a database file or in-memory instance (`sqlite3.connect()`).
2. **Cursor:** Create a cursor object (`conn.cursor()`) to execute SQL commands and fetch result sets.
3. **Parameterized Queries:** Pass parameters using `?` placeholders (never Python string concatenation or f-strings).
4. **Commit & Close:** Persist modifications via `conn.commit()` and close connections via `conn.close()`.

2. End-to-End SQLite CRUD Operations

```
import sqlite3

# Connect to database file
conn = sqlite3.connect("training.db")
cursor = conn.cursor()

# 1. CREATE TABLE
cursor.execute('''
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    marks REAL NOT NULL
)
''')
conn.commit()

# 2. INSERT (Parameterized)
student_records = [
    ("Arham", "arham@cdac.in", 89.5),
    ("Lisa", "lisa@cdac.in", 92.0),
    ("Vinod", "vinod@cdac.in", 97.5)
]
cursor.executemany(
    "INSERT OR IGNORE INTO students (name, email, marks) VALUES (?, ?, ?)",
    student_records
)
```

```
conn.commit()

# 3. SELECT / QUERY
cursor.execute("SELECT id, name, email, marks FROM students WHERE marks >= ?", (90.0,))
top_students = cursor.fetchall()
print("Students with marks >= 90:")
for row in top_students:
    print(f"ID: {row[0]} | Name: {row[1]} | Marks: {row[3]}")

# 4. UPDATE
cursor.execute("UPDATE students SET marks = ? WHERE name = ?", (95.0, "Lisa"))
conn.commit()

# 5. DELETE
cursor.execute("DELETE FROM students WHERE name = ?", ("Arham",))
conn.commit()

# Clean up resources
conn.close()
```

[!IMPORTANT] Always use parameterized queries (`cursor.execute("SELECT * FROM users WHERE email = ?", (email,))`) rather than format strings to protect your application against SQL injection vulnerabilities.